



Behavior-backed GitHub App permissions, in practice

Install from npm. Record one repeatable scenario. Review the exact contract. Then let CI detect permission and coverage drift.

CONSUMER-FIRST WORKFLOW

```
$ pnpm add -D granttrace@beta
$ pnpm exec granttrace record \
  issue-triage -- pnpm test -- issue-triage
$ pnpm exec granttrace check
```

What this edition assumes

You can navigate a terminal, recognize TypeScript, and understand Git at a practical level. The guide focuses on GrantTrace-specific judgment instead of generic coding basics.

NODE 22+ / SCHEMA V2 / GITHUB REST API 2026-03-10

VERIFIED AGAINST NPM AND THE PUBLISHED DOCS / 25 JUL 2026



Read the guide as a system

The useful unit is not a command. It is the full loop from tested behavior to reviewed access.

Fast path

01

Install

Use the published development dependency.

02

Exercise

Run one repeatable GitHub behavior.

03

Record

Observe its supported REST operations.

04

Solve

Map evidence to exact permission alternatives.

05

Review

Decide whether drift is intentional.

06

Enforce

Commit the contract; re-record in CI.

Two lockfiles, two questions

pnpm-lock.yaml

Which exact package versions should this project install?

granttrace.lock.json

Which tested behaviors justify which GitHub App REST permissions?



The claim, precisely

GrantTrace is strongest when its boundary is stated as clearly as its result.



What it proves

- Named scenarios exercised specific, canonical GitHub REST routes.
- Supported evidence maps those routes to exact permission requirements.
- The current recordings reproduce the accepted, deterministic contract.

What remains outside the claim

- Untested paths, background jobs, production-only branches, and missing scenarios.
- GraphQL, Git transport, webhook inference, static analysis, and GitHub Enterprise.
- Whether a permission absent from this contract is safe to remove from production.

The sentence worth keeping

GrantTrace reports the permissions demonstrably required by the named scenarios it successfully recorded. It does not certify whole-application least privilege.



How the repository is shaped

You only need four layers to reason about the implementation.

REPOSITORY MAP

```
src/  
  runtime/ + recorder/  observe supported fetch  
  permissions/         canonicalize + solve  
  contract/            build deterministic schema  
  proof/ + security/   optional live verification  
  cli/                 user-facing workflow  
  
examples/triage-bot/  
  src/triage.ts         product behavior  
  scenario.ts           repeatable observation  
  granttrace.lock.json  accepted result
```

The data path

1 Instrumentation

A managed Node child observes global fetch requests to exactly `api.github.com`.

2 Evidence

Runtime headers and a pinned 49-route documentation catalog must agree.

3 Solver

Exact AND/OR requirements remain intact; non-dominated choices stay visible.

4 Artifact

A stable schema records routes, scenarios, provenance, and selected access.

Package vs repository. Users install `granttrace@beta` from npm. Contributors use `pnpm granttrace` in the checkout; building `dist` is not consumer onboarding.



Start from the npm package

The registry package is the product surface. No global install and no repository build are required.

```
published package / npm registry

$ pnpm add --save-dev
  granttrace@beta
Packages: +14

devDependencies:
+ granttrace 0.1.0-beta.1

Done in 2s using pnpm v10.28.2

$ pnpm exec granttrace --help
GrantTrace 0.1.0-beta.1

Start
  granttrace record <name> --
    <test-command>
```

Captured from a clean consumer project. The npm registry currently maps both beta and latest to 0.1.0-beta.1.

CLIENT	INSTALL	RUN
pnpm	pnpm add -D granttrace@beta	pnpm exec granttrace
npm	npm install -D granttrace@beta	npx granttrace

Keep @beta explicit. It communicates maturity even though npm latest currently points to the same build. Pin the exact version when reproducibility matters more than beta updates.



The triage behavior, not the boilerplate

The included example posts one acknowledgement and validates the returned comment ID.

```
EXAMPLES/TRIAGE-BOT/SRC/TRIAGE.TS
```

```
const result = await request("POST /repos/{owner}/  
  {repo}/issues/{issue_number}/comments", {  
  owner: issue.owner,  
  repo: issue.repository,  
  issue_number: issue.issueNumber,  
  body: "Thanks for the report. ...",  
});  
  
if (!Number.isSafeInteger(result.id) || result.id <= 0)  
  throw new Error("Invalid comment identifier");  
  
return result.id;
```

What matters to GrantTrace

- Behavior: create one issue comment.
- Canonical route: real owner, repository, and issue values become placeholders.
- Requirement: GitHub accepts either `issues:write` or `pull_requests:write` for this route.
- Outcome: the scenario is attributed to whichever route it actually exercised.

Why the example needs no GitHub account

The scenario uses a local HTTP fixture plus `GrantTraceOctokit`. The fixture returns a synthetic ID, while the pinned catalog supplies the permission evidence. No credential or real API mutation is involved.

In a normal app

Standard Octokit requests to `https://api.github.com` use Node's global `fetch` and need no GrantTrace-specific source edit.



Make the scenario boring first

Instrumentation cannot rescue a flaky, broad, or unclear test. Stability comes before recording.

```
consumer project / test first

$ node --import tsx --test
  triage.test.ts

PASS posts a stable triage comment
  to the selected issue

tests      1
suites     0
pass       1
fail       0
cancelled  0
skipped    0
todo       0

duration_ms 438.144625
```

This run used the published package's consumer dependencies in an isolated project. Exact timing varies; the useful signal is one pass and zero failures.

A recordable scenario should be

- named after one user or operational outcome;
- repeatable locally and in CI;
- small enough that every observed route is explainable;
- isolated from production data and credentials.

If the test fails

Stop. GrantTrace classifies a failed, timed-out, or unstartable child as indeterminate evidence, not as proof that a permission is missing.



Record one named behavior

The first successful recording creates private local state and stores a scenario observation.

```
consumer project / record

$ pnpm exec granttrace record
  --no-review triage-comment --
  node --import tsx scenario.ts

GrantTrace initialized
  Local state .granttrace/
              (private, ignored)

Recording started
  Scenario triage-comment
  Timeout 15m

GrantTrace record complete

Scenario triage-comment
Observed 1 GitHub REST operation

Next granttrace check
```

The capture is based on the published CLI running the included triage scenario in an isolated consumer project.

Command anatomy

`record`

run a managed child and observe

`--no-review`

save now; require a final check later

`triage-comment`

stable scenario identity

`--`

everything after this belongs to the child

Replaceable, not historical. Re-recording `triage-comment` replaces its previous successful observation. `.granttrace/` stays private and ignored.



Treat the first diff as a design review

A contract change is a decision point, not an error to dismiss or a file to accept blindly.

```
consumer project / first review

$ pnpm exec granttrace check

GrantTrace contract review required

New permission
  issues: write

Observed in
  Route
    POST /repos/{owner}/{repo}/
      issues/{issue_number}/comments
  Scenario triage-comment
  Evidence Pinned catalog

Next
  Review, then run:
  granttrace check --accept
```

Read four dimensions

- 1 Behavior** Did the scenario intentionally create a comment?
- 2 Route** Is POST .../comments the API operation you expected?
- 3 Access** Does write access match a mutating operation?
- 4 Evidence** Is the pinned catalog an acceptable provenance source here?

Only then accept: `pnpm exec granttrace check --accept` writes the reviewed lock. Never automate acceptance in CI.

What the lockfile actually records

The artifact is deterministic, schema-validated, and deliberately identity-free.

GRANTTRACE.LOCK.JSON / SIMPLIFIED

```
{
  "schemaVersion": 2,
  "toolVersion": "0.1.0-beta.1",
  "apiVersion": "2026-03-10",
  "scenarios": [{ "name": "triage-comment" }],
  "routes": [{
    "method": "POST",
    "template": "/repos/{owner}/{repo}/issues/...",
    "alternatives": [
      [ { "permission": "issues", "level": "write" } ],
      [ { "permission": "pull_requests", "level": "write" } ]
    ],
    "evidence": [ "pinned_catalog" ]
  }],
  "selectedPermissions": { "issues": "write" },
  "permissionFrontier": [ ... ],
  "manualKeeps": {},
  "unknowns": []
}
```

Identity

tool, schema, API, and catalog versions

Attribution

which scenarios exercised each route

Requirements

exact permission alternatives and evidence

Decision

selected access plus full non-dominated frontier

Exceptions

reasoned manual keeps, never mislabeled as observed

Blockers

safe unknown findings that prevent acceptance

Commit boundary

Commit `granttrace.lock.json`. Do not commit or upload `.granttrace/`, including as a CI artifact.



Why issues:write is selected

GitHub permission requirements can contain true alternatives. GrantTrace preserves them instead of flattening them.

ROUTE REQUIREMENT

`issues:write` OR `pull_requests:write`

Both assignments are sufficient. Neither dominates the other.

Permission frontier

The lock retains every non-dominated sufficient assignment. It also chooses one stable assignment for normal review and token requests.

Selection order

- 1 Fewer write permissions
- 2 Lower total access weight
- 3 Fewer distinct permissions
- 4 Lexical order for final stability

This is policy, not mathematical truth

The selected assignment is deterministic and risk-aware. It is not a universal claim that every other valid alternative is worse for every product.

metadata:read

GitHub's mandatory baseline participates in validation but is reported separately from scenario-selected access.



Instrument your own project

Prefer the automatic path. Reach for the explicit adapter only when your transport falls outside it.

A

Standard path / no source edit

A Node child uses global fetch or standard Octokit and targets exactly `https://api.github.com`. GrantTrace injects the recorder preload.

B

Explicit adapter / deliberate source edit

Custom transport, local fixture, or advanced Octokit composition uses GrantTraceOctokit or the exact-version plugin integration.

First scenario checklist

- One user-visible or operational outcome.
- A stable lowercase name such as `issue-triage`.
- A command that performs the same supported REST behavior each run.
- No production data, credentials, or irreversible mutation.
- A failure mode you can distinguish from missing permission.

Origin is a security boundary

Off-origin responses are ignored even if they imitate GitHub's accepted-permissions header. A local fixture contributes observations only through the explicit adapter and pinned catalog.



The daily local loop

Recording is mechanical. Acceptance is a security and coverage review.

- | | | |
|----|----------------|---|
| 01 | Change | Modify behavior or tests. |
| 02 | Test | Prove the scenario remains stable. |
| 03 | Record | Replace its local observation. |
| 04 | Check | Compare the aggregate contract. |
| 05 | Inspect | Read routes, access, evidence, attribution. |
| 06 | Accept | Write the lock only if intentional. |
| 07 | Commit | Review the Git diff with the code change. |

COMMANDS BEHIND THE LOOP

```
$ pnpm exec granttrace record issue-triage -- \
  pnpm test -- issue-triage
$ pnpm exec granttrace check
$ git diff -- granttrace.lock.json
$ pnpm exec granttrace check --accept
$ git add granttrace.lock.json && git commit
```

Review contractions too

Removing a scenario or losing coverage can make the contract smaller. A smaller contract is not automatically safer if the application still performs the unrecorded behavior.



CI detects; humans approve

Reproduce all named scenarios with no intermediate review, then perform one final read-only check.

```
.GITHUB/WORKFLOWS/GRANTTRACE.YML

name: GrantTrace
on: [pull_request]

permissions:
  contents: read

steps:
  - uses: actions/checkout@<reviewed-SHA>
  - uses: actions/setup-node@<reviewed-SHA>
    with: { node-version: 22 }
  - run: corepack enable
  - run: pnpm install --frozen-lockfile
  - run: pnpm exec granttrace record --no-review \
    issue-triage -- pnpm test -- issue-triage
  - run: pnpm exec granttrace record --no-review \
    release-read -- pnpm test -- release-read
  - run: pnpm exec granttrace check
```

1 --no-review

Each intermediate aggregate is intentionally incomplete.

2 final check

The complete set is compared with the committed contract.

3 never --accept

Automation must not approve access or coverage changes.



The automation policy

Package changes, contract drift, and evidence failures have different review paths.

1 Package pin

`pnpm-lock.yaml` fixes the installed npm build. A dependency update can change GrantTrace behavior even when the permission contract does not.

2 Contract pin

`granttrace.lock.json` fixes reviewed routes, scenarios, evidence, and access. A product change can alter it without changing the package version.

Interpret the final check

0 Recordings reproduce the accepted contract.

6 Permission, route, evidence, attribution, or coverage changed.

7 Evidence is unknown, unsupported, malformed, or contradictory.

Exit 6 is review; exit 7 is investigation

For 6, reproduce locally and decide whether the change is intended. For 7, fix the scenario or contribute authoritative catalog evidence. Never guess access to make the workflow pass.

Multiple scenarios

Use `--no-review` on each recording only when one guaranteed final check evaluates the complete aggregate.



Privacy of the artifacts

The committed contract is useful because it excludes identities, commands, and credentials.

ARTIFACT	GIT?	CONTENTS
granttrace.lock.json	Yes	routes, evidence, scenarios, access
.granttrace/observations	No	private per-scenario observations
.granttrace/reports	No	optional identity-free proof results

Never intentionally stored

- raw URLs, query strings, bodies, headers, or rich errors;
- commands, tokens, JWTs, private keys, or broker identifiers;
- owners, repositories, issue numbers, or other resource IDs.

Commit boundary

Commit `granttrace.lock.json`. Keep `.granttrace/` private, ignored, and out of uploaded CI artifacts.

Source provenance

Proof reports include a source commit only when Git reports a clean index and worktree. Dirty or non-Git source is represented as `sourceCommit: null`.



Trust boundaries that remain

GrantTrace reduces stored exposure; it does not make test code or the host machine trustworthy.

A

Recorder boundary

Automatic evidence only comes from exact GitHub.com origin traffic through the supported Node global-fetch path. The recorder is instrumentation, not an OS or network sandbox.

B

Scenario boundary

The test inherits its ordinary environment and can print secrets. Treat the scenario, test runner, runtime, and dependencies as trusted code.

C

Machine and supply-chain boundary

Package checks reduce accidental leakage and packaging mistakes. They cannot eliminate a compromised Node, pnpm, dependency, CI runner, or developer machine.

Manual keeps are honest exceptions

A keep retains intentional, unobserved access with a committed human reason. GrantTrace never labels a manual keep as observed or proven necessary.



Troubleshoot by failure class

Run doctor first, then use the exit code and the first concrete message.

```
consumer project / diagnostics

$ pnpm exec granttrace doctor
GrantTrace doctor

PASS  Node 26.5.0
PASS  .granttrace/ is ignored
PASS  Local state is private;
      1 scenario recording found
INFO  No accepted contract yet
INFO  Optional live proof
      is not configured

Result  Local checks are ready.
```

Any Node version 22 or newer passes. INFO about unconfigured live proof is non-blocking.

CODE	MEANING	CODE	MEANING
0	success	5	unsafe artifact/config
2	invalid usage	6	review required
3	nothing observed	7	bad/unknown evidence
4	child failed/timeout	8	proof/cleanup failure
130	interrupted		

Do not collapse distinct failures

A child failure is not an authorization result. Exit 6 is review. Exit 7 means fix or extend evidence; never guess a permission to make CI green.



Keep live proof separate

Offline record/check is normal. Authenticated proof is optional and fixture-only.

Never target production

Use a dedicated GitHub App, one disposable repository ending in `-granttrace-fixture`, and disposable issue data. The guards reduce accidents; they do not make production mutation safe.

What prove verifies

- exact tool, API, catalog, and accepted schema identity;
- requested access equals scenario-selected access plus manual keeps;
- effective access adds only mandatory metadata:read;
- the token covers exactly one expected repository;
- the scenario reproduces its accepted route/evidence slice;
- applicable negative controls and cleanup behave correctly.

ONLY AFTER DOCS/LIVE-SETUP.MD

```
$ pnpm exec granttrace doctor
$ pnpm exec granttrace prove issue-triage -- \
  pnpm test -- issue-triage
```

Credential providers

Choose exactly one: protected environment secret, owned 0600 file inside an owned 0700 directory, or macOS Keychain.

Recommendation

Do not configure live proof until offline contracts, scenario design, CI drift review, and fixture cleanup are already routine.



Operating card

The smallest useful set of commands, decisions, and source links.

CONSUMER COMMANDS

```
$ pnpm add -D granttrace@beta
$ pnpm exec granttrace doctor
$ pnpm exec granttrace record --no-review \
  issue-triage -- pnpm test -- issue-triage
$ pnpm exec granttrace check
$ git diff -- granttrace.lock.json
$ pnpm exec granttrace check --accept
$ pnpm exec granttrace scenario list
```

Before accepting

- | | |
|--|--|
| ☐ Is the behavior intentional and named accurately? | ☐ Are added and removed routes expected? |
| ☐ Does the permission level fit the operation? | ☐ Is evidence provenance supported and unchanged intentionally? |
| ☐ Did coverage move, disappear, or become broader? | ☐ Would I defend this access change in a security review? |

Primary sources

Documentation: <https://toxfied.github.io/GrantTrace/docs/>

Repository: <https://github.com/ToxFied/GrantTrace>

Package: <https://www.npmjs.com/package/granttrace>

Beta reality

This guide was verified against granttrace@0.1.0-beta.1 on 25 July 2026. Re-check the published docs, npm dist-tags, API version, and limitations before production adoption.